

# SILICON SPRINT HACKATHON

**Round 1 | VLSI Track**

Organized by Maven Silicon

## TECHNICAL REPORT

# DESIGN AND VERIFICATION OF A 4-REQUESTER ROUND-ROBIN ARBITER

Team Name

**BIT LORDS**

Team Head

**Dhananjaya L**

Team Members

Deeksha Manjunath Hegde

Bhanuprakash K A

I M U Chiranth

Tools: Verilog HDL, AMD Vivado 2026.1

Target FPGA: xc7a35tcp236-1

## ABSTRACT

Shared resources such as buses, memories and functional units must be accessed by several requesters, and the logic that decides who is served in each clock cycle is called an arbiter. A round-robin arbiter serves the requesters in a circular order so that no requester can be starved. This report presents the design and verification of a synthesizable 4-requester round-robin arbiter developed for Round 1 of the Silicon Sprint Hackathon organized by Maven Silicon.

The arbiter, named `rr_arbiter`, has a clock, an active-low asynchronous reset and a 4-bit request input. It produces a one-hot (or zero) 4-bit grant and a `grant_valid` flag. An internal 2-bit priority pointer selects the first requester searched in each cycle, and after every successful grant the pointer moves to the requester following the one that was served. The outputs are registered, so a grant appears one clock cycle after the corresponding request. The design is written in plain Verilog-2001 with no delays, no initial blocks for functional behavior, no gated clocks and no inferred latches.

A self-checking Verilog testbench with 12 directed tests and 22 checks was simulated in the Vivado Simulator. The waveform shows 0 errors for the 22 checks. The design was synthesized and implemented in Vivado 2026.1 for the `xc7a35tcpg236-1` device; the implemented netlist contains 28 leaf cells and 34 nets. The timing summary reports no failing endpoints but also states that no user-specified timing constraints exist, so no maximum clock frequency is claimed. The power report gives an estimate of 1.976 W, of which 93% is I/O power, and is treated as a tool estimate rather than a measured value.

**Keywords:** Round-robin arbiter, Verilog HDL, RTL design, self-checking testbench, fairness, Vivado, FPGA, timing, power.

## TABLE OF CONTENTS

| Section    | Title                                  |
|------------|----------------------------------------|
| 1          | Introduction                           |
| 2          | Problem Statement and Requirements     |
| 3          | Architecture and Design Description    |
| 4          | Design Methodology                     |
| 5          | Functional Verification                |
| 6          | Synthesis and Implementation           |
| 7          | Results and Analysis                   |
| 8          | Learning Outcomes                      |
| 9          | Limitations and Scope for Enhancement  |
| 10         | Conclusion                             |
| 11         | References                             |
| Appendix A | RTL Source Code: rr_arbiter.v          |
| Appendix B | Testbench Source Code: tb_rr_arbiter.v |

# 1. INTRODUCTION

## 1.1 Overview

In a digital system, several blocks often need the same resource, for example a bus, a memory port or a processing unit. Only one of them can use the resource at a time, so a block called an arbiter examines the incoming requests and grants access to exactly one requester. The policy used to choose the winner decides how fair the system is. A fixed-priority arbiter always favors the same requester and can starve the others, whereas a round-robin arbiter rotates the priority so that every requester is served in turn.

This project implements a 4-requester round-robin arbiter in Verilog HDL, verifies it with a self-checking testbench, and takes it through the FPGA flow in AMD Vivado 2026.1.

## 1.2 Need for the Project

Arbiters are basic building blocks of interconnects, memory controllers and network-on-chip routers. Designing one exercises several fundamental skills together: finite-state behavior, circular priority handling, synchronous design with an asynchronous reset, and verification of fairness properties.

## 1.3 Project Scope

- Design a synthesizable 4-requester round-robin arbiter in Verilog-2001.
- Grant at most one requester per clock cycle and only a requester whose request is high.
- Rotate priority after every successful grant so that no requester is starved.
- Develop a self-checking testbench covering the required scenarios.
- Run behavioral simulation, synthesis and implementation in Vivado and study the timing and power reports.

# 2. PROBLEM STATEMENT AND REQUIREMENTS

## 2.1 Problem Statement

Design a synthesizable 4-requester round-robin arbiter that controls access to a shared resource. The arbiter shall grant access to at most one requester per clock cycle while ensuring fair access and avoiding starvation. The internal architecture is intentionally left open; the participants choose an RTL architecture that meets all requirements. (Source: Silicon Sprint Hackathon Problem Statement, VLSI, Maven Silicon.)

## 2.2 Module Interface

| Signal      | Width  | Direction | Description                                    |
|-------------|--------|-----------|------------------------------------------------|
| clk         | 1 bit  | Input     | Rising-edge clock                              |
| rst_n       | 1 bit  | Input     | Active-low asynchronous reset                  |
| req         | 4 bits | Input     | Requests from the four requesters              |
| grant       | 4 bits | Output    | One-hot grant, or 0000 when no grant is issued |
| grant_valid | 1 bit  | Output    | High whenever a valid grant is issued          |

## 2.3 Functional Requirements

- After reset, grant = 4'b0000 and grant\_valid = 1'b0, and the initial priority starts at requester 0.
- Starting from the current priority, requesters are searched in circular order and the first active request is granted (for example, priority 2 searches 2, 3, 0, 1).
- At most one grant bit may be set, and a requester is granted only while its req bit is high. If req = 4'b0000, grant = 4'b0000 and grant\_valid = 0.
- After a grant the priority moves to the requester after the granted one (3 wraps to 0). If no grant is issued, the priority is unchanged.
- With req = 4'b1111 the grants rotate in round-robin order, and requests may change every cycle.

## 2.4 Design Constraints

- Pure synthesizable Verilog-2001, with no SystemVerilog constructs.
- No # delays in RTL, no initial blocks for functional behavior, no derived or gated clocks, and no inferred latches.
- Deterministic reset behavior.

# 3. ARCHITECTURE AND DESIGN DESCRIPTION

## 3.1 Architectural Principle

The design has a combinational next-state block and a sequential register block. The combinational block takes the current priority pointer and the request vector and computes the next grant, the next grant\_valid and the next priority. For each of the four priority values, it checks the requests in the circular order that starts at that priority and picks the first active one. The sequential block stores the pointer, the grant and grant\_valid on the rising clock edge and clears them asynchronously when rst\_n goes low.

## 3.2 Priority Pointer and Search Order

| Priority | Search order   | Next priority after grant to R0 / R1 / R2 / R3 |
|----------|----------------|------------------------------------------------|
| 0        | R0, R1, R2, R3 | 1 / 2 / 3 / 0                                  |
| 1        | R1, R2, R3, R0 | 1 / 2 / 3 / 0                                  |
| 2        | R2, R3, R0, R1 | 1 / 2 / 3 / 0                                  |
| 3        | R3, R0, R1, R2 | 1 / 2 / 3 / 0                                  |

The next priority depends only on which requester was granted, as the problem statement requires. When no request is active the combinational block leaves next\_priority equal to the current priority, so the pointer holds its value.

## 3.3 Registered Outputs

Because grant and grant\_valid are registers, they reflect the request pattern that was present at the previous rising clock edge. A request applied between two clock edges is therefore answered one clock cycle later. The testbench is written with this behavior in mind: it changes req, waits for the next rising edge, and checks the outputs 1 ns after the edge.

## 3.4 Elaborated RTL Schematic

The elaborated design from Vivado shows three asynchronous-reset registers (priority\_reg[1:0], grant\_reg[3:0] and grant\_valid\_reg) driven by a network of multiplexers that implement the next\_priority, next\_grant and next\_grant\_valid logic, and confirms the structure described above.

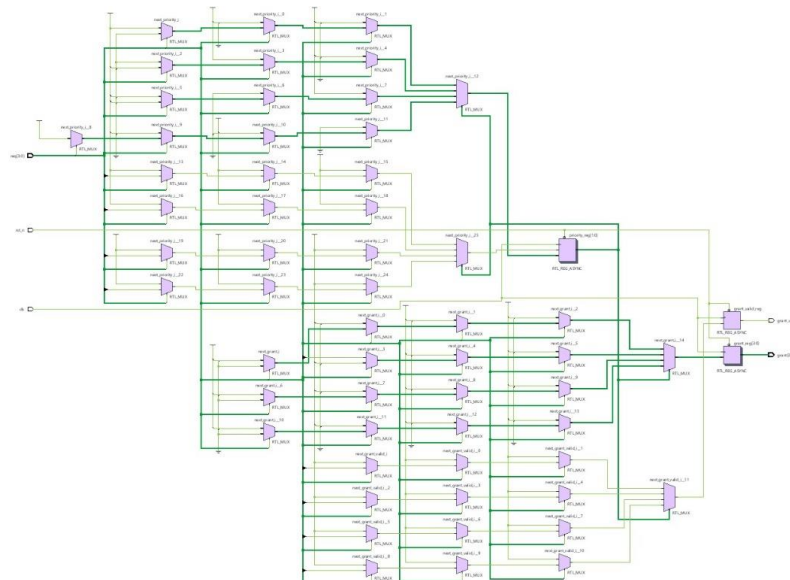


Figure 3.1: Elaborated RTL schematic of rr\_arbiter

## 4. DESIGN METHODOLOGY

### 4.1 Development Tools

| Tool / Technology           | Purpose                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------|
| Verilog HDL (Verilog-2001)  | RTL description of the arbiter and testbench development                          |
| AMD Vivado 2026.1           | Project creation, simulation, RTL analysis, synthesis, implementation and reports |
| Vivado Simulator            | Behavioral functional verification and waveform inspection                        |
| FPGA device xc7a35tcpg236-1 | Target device for implementation                                                  |

### 4.2 Design Flow

1. Study the problem statement and define the interface and priority rules.
2. Write the Verilog module with a combinational next-state block and a registered output block.
3. Write a self-checking testbench with directed tests for every required scenario.
4. Run behavioral simulation and inspect the grant, grant\_valid, errors and checks signals.
5. Open the elaborated design and inspect the RTL schematic.
6. Run synthesis and implementation for xc7a35tcpg236-1.
7. Generate timing and power reports and record the observations and limitations.

## 5. FUNCTIONAL VERIFICATION

### 5.1 Verification Strategy

The testbench (tb\_rr\_arbiter) generates a 10 ns clock, applies directed request patterns and, after each rising clock edge, calls a check\_grant task with the expected grant and grant\_valid. The task counts every check and every error, and in addition tests two general rules on each call: the grant must be one-hot or zero, and no inactive requester may be granted. At the end the testbench prints the total number of checks and errors and a pass or fail message.

## 5.2 Test Cases

| Test | Scenario                          | Stimulus                | Expected grant         |
|------|-----------------------------------|-------------------------|------------------------|
| 1    | Reset                             | rst_n low at start      | 0000, grant_valid = 0  |
| 2    | No requests                       | req = 0000              | 0000, grant_valid = 0  |
| 3    | Single request R0                 | req = 0001              | 0001                   |
| 4    | Single request R1                 | req = 0010              | 0010                   |
| 5    | Single request R2                 | req = 0100              | 0100                   |
| 6    | Single request R3                 | req = 1000              | 1000                   |
| 7    | All requests continuously active  | req = 1111 for 4 cycles | 0001, 0010, 0100, 1000 |
| 8    | Multiple simultaneous requests    | req = 0101 for 2 cycles | 0001, then 0100        |
| 9    | Request withdrawal                | req = 0000, then 0010   | 0000, then 0010        |
| 10   | Priority wrap-around              | req = 0100, then 0011   | 0100, then 0001        |
| 11   | Fairness with persistent requests | req = 1111 for 4 cycles | 0010, 0100, 1000, 0001 |
| 12   | Dynamic request pattern           | req = 1001, 0110, 0100  | 1000, 0010, 0100       |

Test 10 shows the wrap-around rule: after requester 2 is granted the priority becomes 3, and with req = 0011 the search runs 3, 0, 1, so requester 0 is granted. In test 11 the previous grant was requester 0, so the priority is 1 and the four grants follow 1, 2, 3, 0 without any requester being served twice before the others.

## 5.3 Coverage of the Required Scenarios

| Required scenario                            | Covered by                                   |
|----------------------------------------------|----------------------------------------------|
| Reset behavior                               | Test 1                                       |
| No requests                                  | Test 2 and the idle cycle in test 9          |
| Single request                               | Tests 3 to 6                                 |
| Multiple simultaneous requests               | Tests 7, 8 and 12                            |
| All four requests continuously asserted      | Tests 7 and 11                               |
| Changing request patterns                    | Test 12                                      |
| Request withdrawal                           | Test 9                                       |
| Priority wrap-around                         | Test 10 and the last grant in tests 7 and 11 |
| Persistent requests and fairness             | Tests 7 and 11                               |
| Inactive requester granted / multiple grants | Checked inside check_grant on every call     |

## 5.4 Simulation Result

The behavioral simulation ran for about 226 ns. In the waveform the errors counter stays at 00000000 throughout and the checks counter ends at 00000016 (hexadecimal), which is 22

checks, matching the number of check\_grant calls in the testbench. The grant values follow the expected sequences, including the 1, 2, 4, 8 rotation for req = f, the idle period with grant\_valid low when req = 0, and the wrap-around from requester 3 back to requester 0. Each grant appears one clock cycle after the request, as expected from the registered outputs.

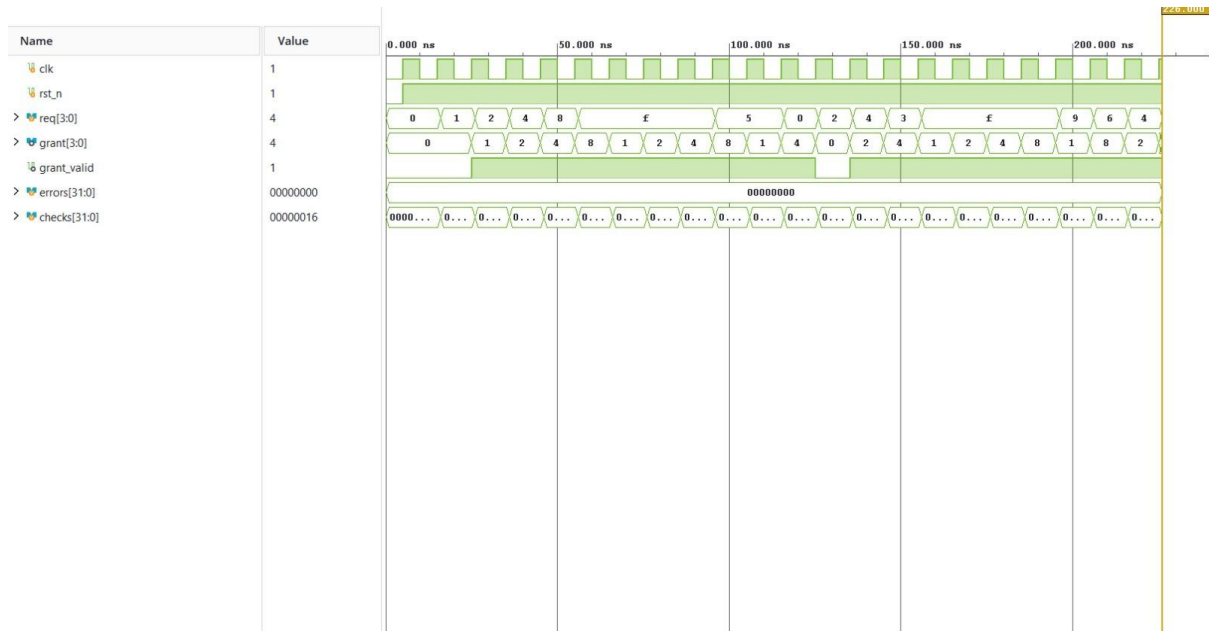


Figure 5.1: Behavioral simulation waveform (0 errors, 22 checks)

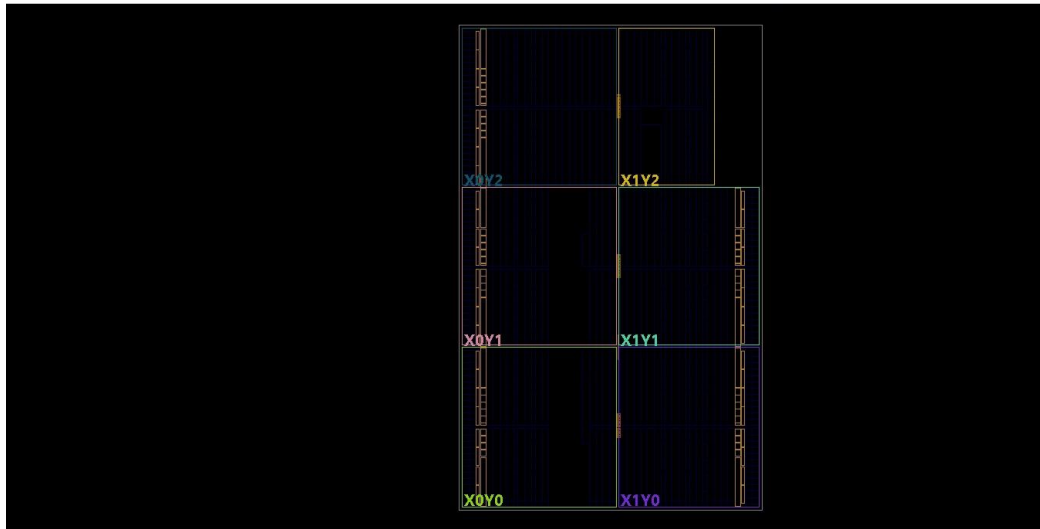
## 6. SYNTHESIS AND IMPLEMENTATION

### 6.1 RTL Elaboration

RTL elaboration converts the Verilog description into a structural representation. The elaborated schematic (Figure 3.1) contains the three asynchronous-reset registers and the multiplexer network for the next-state logic, with the clock and reset feeding the registers.

### 6.2 Synthesis and Implementation

The design was synthesized and implemented for the xc7a35tcpg236-1 device. The implemented netlist of rr\_arbiter contains 28 leaf cells and 34 nets, which is small because the design consists of only seven flip-flops and a limited amount of selection logic. Figure 6.1 shows the device view after implementation, with the placed logic occupying a small region of the fabric.



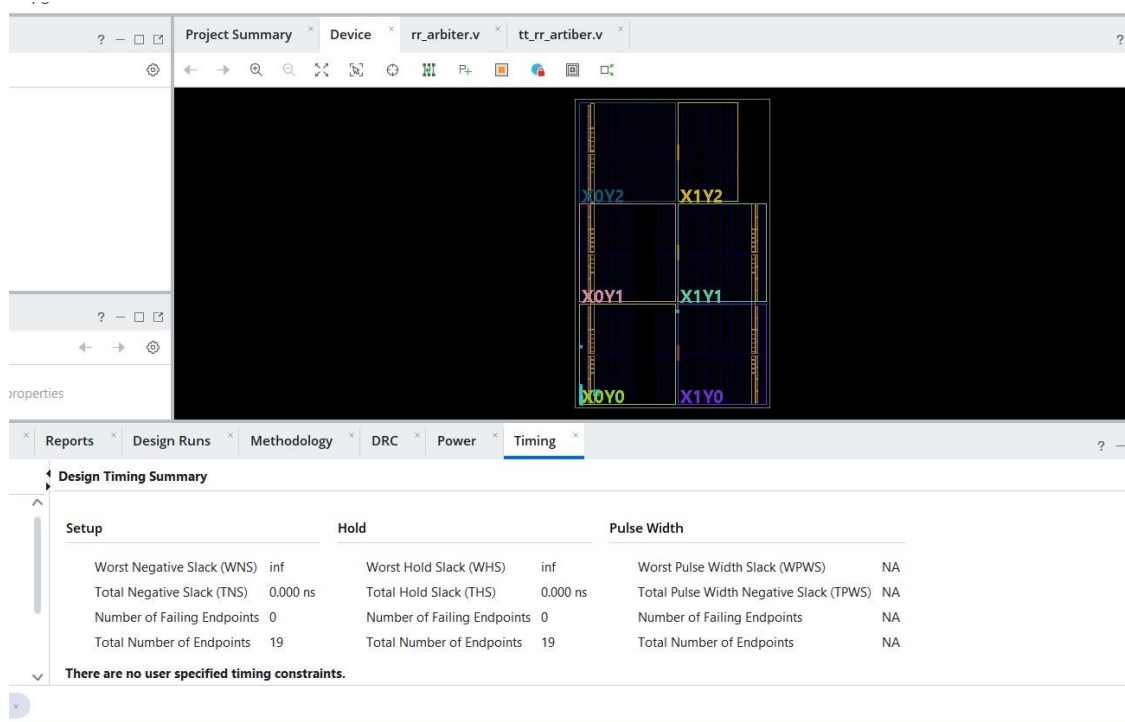
**Figure 6.1: FPGA device view after implementation**

## 7. RESULTS AND ANALYSIS

### 7.1 Timing Analysis

The design timing summary reports 19 endpoints for both setup and hold, with zero failing endpoints, total negative slack and total hold slack of 0.000 ns, and worst setup and hold slack displayed as inf. The pulse width values are not available. The report also states that there are no user specified timing constraints.

Because no clock constraint has been defined, the slack values do not demonstrate that a particular clock period is met, and no maximum operating frequency is claimed for this design. Since the arbiter is a synchronous design, a clock constraint should be added in future work (for example a 10 ns clock on the clk port, matching the simulation clock), together with suitable input and output delays, before making any frequency or timing-closure claim.



**Figure 7.1: Vivado design timing summary**

## 7.2 Power Analysis

The power analysis was run on the implemented netlist. The total on-chip power is reported as 1.976 W and the design power budget is not specified. The breakdown is given below.

| Component           | Power   | Share of total |
|---------------------|---------|----------------|
| Dynamic             | 1.901 W | 96%            |
| Signals             | 0.068 W | 4%             |
| Logic               | 0.059 W | 3%             |
| I/O                 | 1.774 W | 93%            |
| Device static       | 0.075 W | 4%             |
| Total on-chip power | 1.976 W | 100%           |

The I/O component dominates the estimate, while the signal and logic power of the arbiter itself are small. The figures come from Vivado's estimation, with switching activity derived from constraint files, simulation files or vectorless analysis, so they should be treated as a tool estimate and not as measured hardware power. A more reliable figure would need realistic switching activity from simulation and, ideally, measurement on a board.

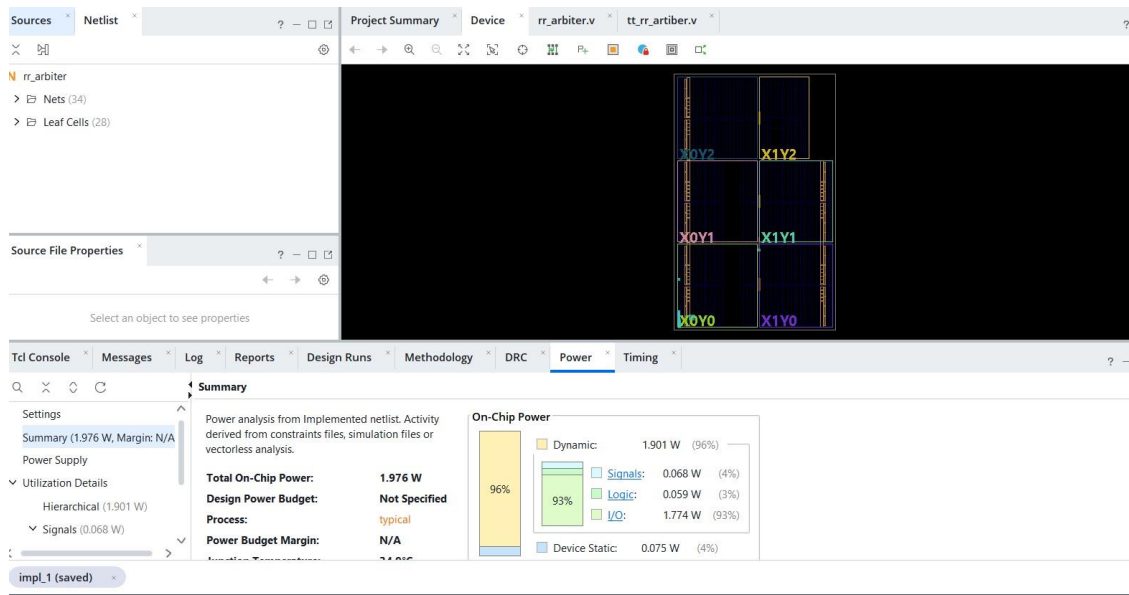


Figure 7.2: Vivado power analysis summary

### 7.3 Overall Result

| Project requirement                              | Status                    | Evidence                             |
|--------------------------------------------------|---------------------------|--------------------------------------|
| 4-requester round-robin arbiter in Verilog-2001  | Completed                 | rr_arbiter.v, RTL schematic          |
| One-hot grant, granted only to active requesters | Verified                  | check_grant, 22 checks, 0 errors     |
| Priority rotation and wrap-around                | Verified                  | Tests 7, 10, 11 and waveform         |
| Fairness with persistent requests                | Verified                  | Tests 7 and 11                       |
| Reset behavior                                   | Verified                  | Test 1                               |
| Self-checking testbench                          | Completed                 | tb_rr_arbiter.v, errors = 0          |
| Synthesis and implementation                     | Completed                 | Implemented netlist, device view     |
| Timing analysis                                  | Generated with limitation | No user-specified timing constraints |
| Power analysis                                   | Generated with limitation | Estimate on implemented netlist      |

## 8. LEARNING OUTCOMES

- Understanding how round-robin arbitration achieves fairness and avoids starvation.
- Writing synthesizable Verilog-2001 with a combinational next-state block and registered outputs, without inferred latches.
- Using an active-low asynchronous reset correctly in a sequential design.
- Developing a self-checking testbench with an expected-value task and general rule checks.

- Reading simulation waveforms and relating the output timing to the registered design.
- Running the Vivado flow from elaboration through implementation and interpreting timing and power reports.
- Recognizing that timing results without constraints, and power estimates without realistic activity, must not be presented as performance claims.

## 9. LIMITATIONS AND SCOPE FOR ENHANCEMENT

### 9.1 Current Limitations

- The testbench uses directed tests with hard-coded expected values. It has no independent reference model, so the internal priority pointer is checked only indirectly through the grant sequence.
- The asynchronous reset is tested only at the start of simulation, not in the middle of operation.
- No timing constraints were defined, so no maximum frequency or timing closure is claimed.
- The power value is a tool estimate with incomplete activity information, dominated by I/O power.
- The design was not validated on a physical FPGA board.

### 9.2 Future Enhancements

- Add a reference model to the testbench and compare grant and priority every cycle, and add long constrained-random request sequences.
- Add a starvation check that confirms every persistent requester is served within four grants, and a reset-in-the-middle test.
- Define a clock constraint and input and output delays, then report real setup and hold slack.
- Rename the priority signal (a reserved word in SystemVerilog) to improve portability across tools.
- Parameterize the number of requesters, and explore weighted or masked variants of the arbiter.
- Implement and validate the design on an FPGA development board.

## 10. CONCLUSION

This project designed and verified a synthesizable 4-requester round-robin arbiter in Verilog-2001. The arbiter grants at most one active requester per cycle, rotates the priority after each grant, wraps correctly from requester 3 to requester 0 and resets deterministically. A self-checking testbench with 12 directed tests and 22 checks completed with zero errors in the

Vivado Simulator, and the design was synthesized and implemented for the xc7a35tcpg236-1 device with a small netlist of 28 leaf cells.

The timing and power reports also gave useful lessons: without clock constraints no frequency should be claimed, and a power estimate based on incomplete activity is not a measurement. The design provides a solid base for a more thoroughly verified, constrained and board-validated arbiter.

## **11. REFERENCES**

- [1] Maven Silicon, "Silicon Sprint Hackathon: Problem Statement - VLSI."
- [2] AMD Vivado Design Suite documentation and in-tool reports used during the project.
- [3] Verilog HDL (IEEE 1364-2001) concepts used for synthesizable RTL and testbench development.

## APPENDIX A: RTL SOURCE CODE (rr\_arbiter.v)

```
module rr_arbiter (
    input      clk,
    input      rst_n,
    input  [3:0] req,
    output reg [3:0] grant,
    output reg      grant_valid
);

    // Priority pointer: 0, 1, 2, or 3
    reg [1:0] priority;

    reg [1:0] next_priority;
    reg [3:0] next_grant;
    reg      next_grant_valid;

    // Combinational arbitration logic
    always @(*) begin

        // Default values
        next_grant      = 4'b0000;
        next_grant_valid = 1'b0;
        next_priority   = priority;

        // Round-robin search
        case (priority)

            2'd0: begin
                if (req[0]) begin
                    next_grant      = 4'b0001;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd1;
                end
                else if (req[1]) begin
                    next_grant      = 4'b0010;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd2;
                end
                else if (req[2]) begin
                    next_grant      = 4'b0100;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd3;
                end
                else if (req[3]) begin
                    next_grant      = 4'b1000;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd0;
                end
            end

            2'd1: begin
                if (req[1]) begin
                    next_grant      = 4'b0010;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd2;
                end
                else if (req[2]) begin
                    next_grant      = 4'b0100;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd3;
                end
                else if (req[3]) begin
                    next_grant      = 4'b1000;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd0;
                end
                else if (req[0]) begin
                    next_grant      = 4'b0001;
                    next_grant_valid = 1'b1;
                    next_priority   = 2'd1;
                end
            end

            2'd2: begin
                if (req[2]) begin
                    next_grant      = 4'b0100;
                    next_grant_valid = 1'b1;
                end
            end
        end
    end
```

```

        next_priority    = 2'd3;
    end
    else if (req[3]) begin
        next_grant        = 4'b1000;
        next_grant_valid = 1'b1;
        next_priority     = 2'd0;
    end
    else if (req[0]) begin
        next_grant        = 4'b0001;
        next_grant_valid = 1'b1;
        next_priority     = 2'd1;
    end
    else if (req[1]) begin
        next_grant        = 4'b0010;
        next_grant_valid = 1'b1;
        next_priority     = 2'd2;
    end
end

2'd3: begin
    if (req[3]) begin
        next_grant        = 4'b1000;
        next_grant_valid = 1'b1;
        next_priority     = 2'd0;
    end
    else if (req[0]) begin
        next_grant        = 4'b0001;
        next_grant_valid = 1'b1;
        next_priority     = 2'd1;
    end
    else if (req[1]) begin
        next_grant        = 4'b0010;
        next_grant_valid = 1'b1;
        next_priority     = 2'd2;
    end
    else if (req[2]) begin
        next_grant        = 4'b0100;
        next_grant_valid = 1'b1;
        next_priority     = 2'd3;
    end
end

endcase
end

// Sequential logic
// Active-low asynchronous reset
always @(posedge clk or negedge rst_n) begin

    if (!rst_n) begin
        priority    <= 2'd0;
        grant        <= 4'b0000;
        grant_valid <= 1'b0;
    end
    else begin
        priority    <= next_priority;
        grant        <= next_grant;
        grant_valid <= next_grant_valid;
    end
end

end
endmodule

```

## APPENDIX B: TESTBENCH SOURCE CODE (tb\_rr\_arbiter.v)

```
`timescale 1ns/1ps

module tb_rr_arbiter;

    reg        clk;
    reg        rst_n;
    reg [3:0]   req;

    wire [3:0]  grant;
    wire        grant_valid;

    integer errors;
    integer checks;

    // DUT
    rr_arbiter dut (
        .clk(clk),
        .rst_n(rst_n),
        .req(req),
        .grant(grant),
        .grant_valid(grant_valid)
    );

    // Clock: 10 ns period
    always #5 clk = ~clk;

    // -----
    // Check expected grant
    // -----
    task check_grant;
        input [3:0] expected_grant;
        input        expected_valid;

        begin
            checks = checks + 1;

            if (grant != expected_grant) begin
                $display("ERROR: req=%b grant=%b expected=%b",
                    req, grant, expected_grant);
                errors = errors + 1;
            end

            if (grant_valid != expected_valid) begin
                $display("ERROR: req=%b grant_valid=%b expected=%b",
                    req, grant_valid, expected_valid);
                errors = errors + 1;
            end

            // Grant must be one-hot or zero
            if ((grant != 4'b0000) &&
                (grant != 4'b0001) &&
                (grant != 4'b0010) &&
                (grant != 4'b0100) &&
                (grant != 4'b1000)) begin

                $display("ERROR: Grant is not one-hot: %b", grant);
                errors = errors + 1;
            end

            // Grant cannot be given to inactive requester
            if ((grant & ~req) != 4'b0000) begin
                $display("ERROR: Inactive requester granted. req=%b grant=%b",
                    req, grant);
                errors = errors + 1;
            end
        end
    endtask

    // -----
    // Main test sequence
    // -----
    initial begin
```

```

clk      = 1'b0;
rst_n    = 1'b0;
req      = 4'b0000;
errors   = 0;
checks   = 0;

$display("=====");
$display(" Silicon Sprint Round-Robin Arbiter Test ");
$display("=====");

// -----
// TEST 1: Reset
// -----
#2;

if ((grant !== 4'b0000) || (grant_valid !== 1'b0)) begin
    $display("ERROR: Reset behavior failed");
    errors = errors + 1;
end

$display("TEST 1: Reset                PASS");

// Release reset
#3;
rst_n = 1'b1;

// -----
// TEST 2: No requests
// -----
req = 4'b0000;

@(posedge clk);
#1;
check_grant(4'b0000, 1'b0);

$display("TEST 2: No requests          PASS");

// -----
// TEST 3: Single requester 0
// -----
req = 4'b0001;

@(posedge clk);
#1;
check_grant(4'b0001, 1'b1);

$display("TEST 3: Single request R0    PASS");

// -----
// TEST 4: Single requester 1
// -----
req = 4'b0010;

@(posedge clk);
#1;
check_grant(4'b0010, 1'b1);

$display("TEST 4: Single request R1    PASS");

// -----
// TEST 5: Single requester 2
// -----
req = 4'b0100;

@(posedge clk);
#1;
check_grant(4'b0100, 1'b1);

$display("TEST 5: Single request R2    PASS");

// -----
// TEST 6: Single requester 3
// -----
req = 4'b1000;

```

```

@(posedge clk);
#1;
check_grant(4'b1000, 1'b1);

$display("TEST 6: Single request R3      PASS");

// -----
// TEST 7: All requesters continuously active
// Expected: 0 -> 1 -> 2 -> 3
// -----
req = 4'b1111;

@(posedge clk);
#1;
check_grant(4'b0001, 1'b1);

@(posedge clk);
#1;
check_grant(4'b0010, 1'b1);

@(posedge clk);
#1;
check_grant(4'b0100, 1'b1);

@(posedge clk);
#1;
check_grant(4'b1000, 1'b1);

$display("TEST 7: Continuous requests   PASS");

// -----
// TEST 8: Multiple requests
// Current priority should be 0 after R3
// req = R0 + R2
// Expected R0
// -----
req = 4'b0101;

@(posedge clk);
#1;
check_grant(4'b0001, 1'b1);

// Now priority = 1
// Search 1 -> 2 -> 3 -> 0
// req = R0 + R2
// Expected R2
@(posedge clk);
#1;
check_grant(4'b0100, 1'b1);

$display("TEST 8: Multiple requests     PASS");

// -----
// TEST 9: Request withdrawal
// -----
req = 4'b0000;

@(posedge clk);
#1;
check_grant(4'b0000, 1'b0);

// R1 request
req = 4'b0010;

@(posedge clk);
#1;
check_grant(4'b0010, 1'b1);

$display("TEST 9: Request withdrawal    PASS");

// -----
// TEST 10: Wrap-around
// Force priority to 3 by granting R2

```

```

// Current priority after R1 is 2
// -----
req = 4'b0100;

@(posedge clk);
#1;
check_grant(4'b0100, 1'b1);

// Now priority = 3
// req = R0 + R1
// Search: 3 -> 0 -> 1 -> 2
// Expected R0
req = 4'b0011;

@(posedge clk);
#1;
check_grant(4'b0001, 1'b1);

$display("TEST 10: Priority wrap-around PASS");

// -----
// TEST 11: Fairness
// All requesters continuously active
// Expected 1 -> 2 -> 3 -> 0
// because previous grant was R0
// -----
req = 4'b1111;

@(posedge clk);
#1;
check_grant(4'b0010, 1'b1);

@(posedge clk);
#1;
check_grant(4'b0100, 1'b1);

@(posedge clk);
#1;
check_grant(4'b1000, 1'b1);

@(posedge clk);
#1;
check_grant(4'b0001, 1'b1);

$display("TEST 11: Fairness          PASS");

// -----
// TEST 12: Dynamic request pattern
// -----
req = 4'b1001;

@(posedge clk);
#1;
check_grant(4'b1000, 1'b1);

req = 4'b0110;

@(posedge clk);
#1;
check_grant(4'b0010, 1'b1);

req = 4'b0100;

@(posedge clk);
#1;
check_grant(4'b0100, 1'b1);

$display("TEST 12: Dynamic requests    PASS");

// -----
// FINAL RESULT
// -----
$display("-----");
$display("Total checks = %0d", checks);

```

```

$display("Total errors = %0d", errors);

if (errors == 0) begin
    $display("=====");
    $display(" ALL TESTS PASSED - 0 ERRORS");
    $display("=====");
end
else begin
    $display("=====");
    $display(" TEST FAILED - %0d ERRORS", errors);
    $display("=====");
end

$finish;
end

endmodule

```